



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

实验报告

实验二：共享单车需求预测

姓名：孙沐赞

学号：2311534

专业：计算机科学与技术

2025 年 10 月 24 日

一、实验要求

1. **实验目的：**用环境因素预测共享单车每小时租赁量。
2. **初级要求：**
 - (i) 实现完整的多元线性回归流程，包括数据预处理（分类变量编码、数值标准化、异常值处理等）与 7:3 时间序列划分。
 - (ii) 分别使用批量梯度下降（BGD）和随机梯度下降（SGD）训练模型，通过对比不同学习率的损失收敛曲线寻找最优参数，并评估模型的 MSE/MAE。
3. **中级要求：**分析模型结果，包括排序特征重要性、从业务角度解读模型规律（如天气影响），并计算特征的统计显著性（P 值）及探索交互作用。
4. **高级要求：**实现并对比岭回归与 LASSO 回归，使用交叉验证选择超参数，通过模型诊断（如残差分析、正态性检验）和时间序列可视化来深度分析预测效果与特征筛选情况。

二、实验原理

（一）多元线性回归与优化原理

- **多元线性回归 (MLR):** 核心思想是建立一个线性模型，将目标变量（每小时租赁量 y ）预测为多个输入特征（如温度、湿度）的加权和。模型的目标是找到最优的参数（权重）向量 θ ，使得预测值 $\hat{y}$ 与真实值 y 之间的误差最小。其数学表达式为：

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n = \mathbf{X} \cdot \theta$$

其中 $\hat{y}$ 是预测租赁量， $\mathbf{X}$ 是特征矩阵（包含偏置项 $x_0 = 1$ ）， θ 是模型参数。

- **批量梯度下降 (BGD):** BGD 是一种优化算法，用于最小化成本函数 $J(\theta)$ （通常为均方误差 MSE）。它在每次迭代时，会计算整个训练集（ m 个样本）的梯度，然后沿着梯度的反方向更新参数 θ 。这确保了收敛路径平稳，但计算成本高。参数更新公式为：

$$\theta_j := \theta_j - \alpha \cdot \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}$$

其中 α 是学习率， $\hat{y}^{(i)}$ 是第 i 个样本的预测值。

- **随机梯度下降 (SGD):** 与 BGD 不同，SGD 在每次迭代时仅使用单个训练样本 $(x^{(i)}, y^{(i)})$ 来估计梯度并更新参数 θ 。这使得 SGD 的训练速度快得多，并且其噪声特性有助于跳出局部最优解，但收敛路径会非常振荡。其更新规则（针对单一样本 i ）为：

$$\theta_j := \theta_j - \alpha \cdot (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}$$

（二）性能评价指标

- **均方误差 (Mean Squared Error, MSE):** 衡量预测值与真实值之间误差的平方均值。该指标对异常值更为敏感，值越小表示模型预测越精确：

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$$

其中：

- m 为样本总数;
 - $y^{(i)}$ 为第 i 个样本的真实租赁量;
 - $\hat{y}^{(i)}$ 为第 i 个样本的预测租赁量。
- **平均绝对误差 (Mean Absolute Error, MAE):** 衡量预测值与真实值之间绝对误差的平均值。该指标在业务上更易于解释, 且对异常值的鲁棒性强于 MSE:

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |y^{(i)} - \hat{y}^{(i)}|$$

其中 $m, y^{(i)}, \hat{y}^{(i)}$ 的定义同上。

三、 初级要求实现

(一) 数据加载函数 (load_data)

此函数负责从磁盘读取 CSV 数据文件。

```

1  def load_data(path):
2      """按时间顺序读取CSV数据并返回DataFrame。"""
3      df = pd.read_csv(path)
4      df = df.sort_values(['dteday', 'hr']).reset_index(drop=True)
5      return df

```

上述代码利用 ‘pandas’ 库的 ‘read_csv’ 函数加载数据。核心步骤是调用 ‘sort_values’ 方法, 通过 ‘dteday’ (日期) 和 ‘hr’ (小时) 两个键进行排序, 这确保了数据严格按照时间先后顺序排列。最后, 通过 ‘reset_index’ 重置索引, 保证后续按索引切分数据的正确性。

(二) 时间序列划分函数 (time_split)

此函数负责将数据集按时间顺序分割为训练集和测试集。

```

1  def time_split(df, train_frac=0.7):
2      """按时间顺序切分数据集, 避免数据泄漏。"""
3      n = len(df)
4      split = int(n * train_frac)
5      train = df.iloc[:split].reset_index(drop=True)
6      test = df.iloc[split:].reset_index(drop=True)
7      return train, test

```

该函数首先获取数据集总长度 n , 并根据设定的训练集比例 (‘train_frac=0.7’) 计算出分割点 ‘split’。我们再使用 ‘iloc’ 索引器, 将索引在 ‘split’ 之前的数据 (即时间上较早的 70% 数据) 划分为训练集 ‘train’, 将 ‘split’ 之后的数据 (时间上较晚的 30% 数据) 划分为测试集 ‘test’。

(三) 数据预处理主函数 (preprocess)

在数据预处理的过程中, 我们整合了从特征工程到标准化的所有步骤。数据预处理函数 (preprocess) 函数首先从数据帧 ‘df’ 中提取目标变量 ‘cnt’ (即租赁量), 将其转换为浮点型 ‘numpy’ 数组, 并利用 ‘reshape’ 方法将其重塑为 ‘(-1, 1)’ 维度的列向量 y , 以符合后续模型训练的格式要求。

```
1 y = df['cnt'].astype(float).to_numpy().reshape(-1, 1)
```

该函数对特征矩阵 $\mathbf{X}$ 执行一系列转换，具体如下所述。

1. 连续变量处理

我们在 ‘preprocess’ 函数中定义了 ‘continuous_cols’ 列表，执行缺失值、异常值和标准化三个关键步骤。

```
1 # 1. 定义连续特征
2 continuous_cols = ['temp', 'atemp', 'hum', 'windspeed']
3
4 # 3. 缺失值处理 (连续)
5 for c in continuous_cols:
6     if X[c].isnull().any():
7         X[c] = X[c].fillna(X[c].median())
8
9 # 4. 异常值处理 (裁剪)
10 for col in continuous_cols:
11     lower = X[col].quantile(0.01)
12     upper = X[col].quantile(0.99)
13     X[col] = X[col].clip(lower, upper)
14
15 # 6. 标准化连续特征
16 scaler_used = scaler if scaler is not None else CustomScaler()
17 cols_to_scale = [c for c in continuous_cols if c in X.columns]
18 if cols_to_scale:
19     if fit_scaler:
20         X[cols_to_scale] = scaler_used.fit_transform(X[cols_to_scale])
21     else:
22         X[cols_to_scale] = scaler_used.transform(X[cols_to_scale])
```

上述代码首先定义了 ‘continuous_cols’ 列表(包含 ‘temp’, ‘atemp’ 等)。然后使用 ‘fillna(X[c].median())’ 方法将所有缺失值替换为该特征的中位数，这种方法对异常值具有较好的鲁棒性。之后我们进行代码执行异常值处理：通过 ‘quantile’ 方法计算每个特征的 1% 和 99% 分位数(‘lower’, ‘upper’)，并调用 ‘clip’ 函数将所有超出此范围的极端值“裁剪”到这两个分位数边界上。最后，代码调用 ‘CustomScaler’ 类对这些连续特征进行 Z-score 标准化(CustomScaler 的功能在后续内容中会讲到，主要实现了对数据的标准化操作)。

2. 离散变量处理

对于离散(分类)变量，我们先定义其基础列表，再进行缺失值填充和独热编码。

```
1 # 1. 定义分类特征
2 categorical_cols = ['season', 'yr', 'holiday', 'workingday', '
3     weathersit']
4
5 # 3. 缺失值处理 (分类)
6 # (注意：此处的 categorical_cols 尚未包含 time_period)
```

```

6         for c in categorical_cols:
7             if X[c].isnull().any():
8                 X[c] = X[c].fillna(X[c].mode().iloc[0])
9
10        # 5. 对分类列进行 one-hot 编码
11        # (注意: 此处的 categorical_cols 已在 "小时处理" 步骤中被追加了
           time_period)
12        if categorical_cols:
13            X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

```

上述代码首先定义了基础的 ‘categorical_cols’ 列表（包含 ‘season’, ‘yr’ 等），但是其中不包含 weekday，因为星期的信息已经被 workingday 与 holiday 包含，如果引入 weekday，则会增加模型的训练负担。再使用 ‘fillna(X[c].mode().iloc[0])’ 将分类变量中的缺失值替换为该特征的众数。处理完缺失值后，代码调用 ‘pandas’ 库中核心的 ‘get_dummies’ 函数来执行独热编码（One-Hot Encoding）。‘drop_first=True’ 是一个关键参数，它会在转换时自动丢弃每个特征的第一个类别，以此来有效避免“虚拟变量陷阱”所导致的多重共线性问题。

（四） 对小时 (hr) 的特征工程

在预处理中，‘hr’（小时）特征是一个关键的时间变量。如果将其（0-23 共 24 个值）直接进行独热编码，会产生 24 个新特征，这给模型的训练带来很大的负担。因此，我们采用了一种“三级划分法”的特征工程策略，在保留小时数据特征的前提下，更清晰地描述其业务含义，以减少模型的干扰因素。

该策略的核心是 ‘map_hour_to_3_periods’ 函数，其定义如下：

```

1     def map_hour_to_3_periods(hour):
2         if hour in [7, 8, 9, 16, 17, 18, 19]:
3             return 'peak'
4         elif hour in [10, 11, 12, 13, 14, 15, 20]:
5             return 'off_peak'
6         else:
7             return 'low_hours'

```

该函数通过一系列 ‘if/elif/else’ 条件判断，将输入的小时 ‘hour’ 映射为字符串标签。它将早晚通勤高峰时段定义为 ‘peak’（高峰期），将白天的常规时段定义为 ‘off_peak’（平峰期），而将其余的深夜和凌晨时段定义为 ‘low_hours’（低谷期）。

在 ‘preprocess’ 主函数中，该函数通过 ‘apply’ 方法被调用，以执行特征的转换和替换：

```

1         # 2. 选取特征，并创建新的 'time_period' 特征
2         features_needed = continuous_cols + categorical_cols + ['hr']
3         X = df[features_needed].copy()
4
5         X['time_period'] = X['hr'].apply(map_hour_to_3_periods)
6         X = X.drop('hr', axis=1)
7
8         categorical_cols.append('time_period')

```

上述代码是 ‘preprocess’ 函数中承上启下的关键步骤。我们首先从 ‘df’ 中复制所有需要的特征（包括 ‘hr’）。接着，它调用 ‘map_hour_to_3_periods’ 函数，将数值型的 ‘hr’ 列转换为 ‘time_period’

分类特征，原始的 ‘hr’ 列被丢弃，并将新特征名 ‘time_period’ 追加到 ‘categorical_cols’ 列表中，以确保能在后续的离散变量处理步骤中被正确地进行独热编码。

（五） 自定义标准化类 (CustomScaler)

为避免引入 ‘scikit-learn’ 库，我们手动实现了 ‘StandardScaler’ 的功能。

```

1      class CustomScaler:
2          def __init__(self):
3              self.mean_ = None
4              self.std_ = None
5
6          def fit_transform(self, data):
7              self.mean_ = np.mean(data, axis=0)
8              self.std_ = np.std(data, axis=0)
9              # 防止除以零
10             self.std_[self.std_ == 0] = 1.0
11             return (data - self.mean_) / self.std_
12
13         def transform(self, data):
14             if self.mean_ is None or self.std_ is None:
15                 raise RuntimeError("Scaler has not been fitted yet. Call fit_transform first.")
16             return (data - self.mean_) / self.std_

```

该类包含了三个方法：‘__init__’ 用于初始化存储均值 ‘mean_’ 和标准差 ‘std_’ 的变量。‘fit_transform’ 方法被设计为仅在训练集上调用，它计算训练数据的均值和标准差，将它们存储在实例变量中，并返回标准化后的训练数据。‘transform’ 方法被设计为仅在测试集上调用，它使用已存储的（来自训练集的）均值和标准差来标准化测试数据。如果我们将训练集和测试集合并在一起计算 μ 和 σ ，或者单独计算测试集的 μ 和 σ ，就等同于在训练阶段“偷看”了未来的数据。这会导致模型在测试集上的表现被高估，使其在的预测能力（泛化能力）远低于预期。

（六） 核心计算方法

1. 正规方程（闭式解）计算线性回归参数

正规方程（Normal Equation）提供了一种求解线性回归参数 θ 的解析解（闭式解），它无需像梯度下降一样设置学习率或进行迭代。其核心是直接求解使成本函数 $J(\theta)$ 梯度为零的 θ 值。

```

1      def normal_equation(X, y):
2          """使用正规方程（闭式解）计算线性回归参数。"""
3          try:
4              theta = np.linalg.inv(X.T.dot(X)).dot(X.T).dot(y)
5          except np.linalg.LinAlgError:
6              theta = np.linalg.pinv(X.T.dot(X)).dot(X.T).dot(y)
7          return theta

```

上述代码实现了 $\theta = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ 的计算。代码首先计算 $\mathbf{X}^T \mathbf{X}$ ，然后尝试使用 ‘np.linalg.inv’ 计算其逆矩阵。‘try...except’ 块用于处理特殊情况：如果 $\mathbf{X}^T \mathbf{X}$ 是奇异矩阵（不可逆），程序将捕

获 ‘LinAlgError’ 异常，并转而使用 ‘np.linalg.pinv’ 计算其“伪逆”，从而确保模型在特征高度相关或特征数量大于样本数量时依然能够返回一个有效的解。

2. 批量梯度下降（BGD）计算线性回归参数

批量梯度下降（BGD）是一种迭代优化算法，它在每一步更新时都会使用全部训练样本来计算梯度的精确平均值。

```

1  def batch_gradient_descent(X, y, lr=0.01, epochs=1000, theta_init=
    None, verbose=False):
2      """批量梯度下降（BGD）。"""
3      n, m = X.shape
4      theta = np.zeros((m, 1)) if theta_init is None else
        theta_init.copy()
5      losses = []
6      for epoch in range(epochs):
7          mse, grad = compute_loss(X, y, theta)
8          theta -= lr * grad
9          losses.append(mse)
10         # ... (省略 verbose 打印) ...
11     return theta, losses

```

上述代码首先初始化参数 θ 和一个用于记录损失的列表 ‘losses’。在 ‘epochs’ 循环中，它调用 ‘compute_loss’ 函数（该函数内部计算了在整个数据集 ‘X’ 上的 MSE 损失和梯度 ‘grad’）。之后，代码执行核心更新步骤 ‘theta -= lr * grad’，即让参数 θ 沿着梯度的反方向（使损失减小最快的方向）移动一小步，步长由学习率 ‘lr’ 控制。每次迭代的损失 ‘mse’ 被追加到 ‘losses’ 列表中，用于后续绘制收敛曲线。

‘compute_loss’ 函数是用于在每一次迭代中，根据整个训练集计算当前的损失（MSE）以及损失函数相对于参数 θ 的梯度。

```

1  def compute_loss(X, y, theta):
2      """计算MSE损失和梯度。"""
3      n = X.shape[0]
4      errors = X.dot(theta) - y
5      mse = np.mean(errors ** 2)
6      grad = (2.0 / n) * X.T.dot(errors)
7      return mse, grad

```

上述代码首先通过 ‘X.dot(theta)’ 进行矩阵乘法，一步到位地计算出当前 θ 下对 * 所有 * 训练样本的预测值 $\hat{y}$ 。接着，它计算预测值 $\hat{y}$ 与真实值 y 之间的差值，得到 ‘errors’ 向量（即残差向量 $\hat{y} - y$ ）。‘mse = np.mean(errors ** 2)’ 这行代码计算了残差向量中所有元素平方后的均值，这正是均方误差（MSE）的定义，即 $J(\theta)$ 。最后一行代码 ‘grad = (2.0 / n) * X.T.dot(errors)’ 是 MSE 成本函数 $J(\theta) = \frac{1}{n} \sum (\hat{y} - y)^2$ 关于 θ 求导的向量化实现 $\nabla J(\theta) = \frac{2}{n} \mathbf{X}^T (\mathbf{X}\theta - y)$ 。函数最终同时返回计算得到的 ‘mse’ 标量和 ‘grad’ 梯度向量，前者用于监控收敛过程，后者用于 BGD 的参数更新。

3. 随机梯度下降（SGD）计算线性回归参数

随机梯度下降（SGD）是 BGD 的一种变体，它在每次迭代时仅使用单个随机样本来估计梯度，这使得训练速度极快，但收敛过程充满噪声。

```

1  def stochastic_gradient_descent(X, y, lr=0.01, epochs=10, theta_init=
    None, verbose=False):
2      """随机梯度下降 (SGD) 。"""
3      n, m = X.shape
4      theta = np.zeros((m, 1)) if theta_init is None else
        theta_init.copy()
5      epoch_losses = []
6      for epoch in range(epochs):
7          perm = np.random.permutation(n)
8          X_shuf, y_shuf = X[perm], y[perm]
9          for i in range(n):
10             xi = X_shuf[i:i+1]
11             yi = y_shuf[i:i+1]
12             error = xi.dot(theta) - yi
13             grad = 2.0 * xi.T.dot(error)
14             theta -= lr * grad
15
16             mse = np.mean((X.dot(theta) - y) ** 2)
17             epoch_losses.append(mse)
18             # ... (省略 verbose 打印) ...
19         return theta, epoch_losses

```

上述代码的外层循环 ‘for epoch in range(epochs)’ 表示对数据集的完整遍历次数。在每个 ‘epoch’ 开始时，代码通过 ‘np.random.permutation(n)’ 打乱训练集顺序，以确保随机性。内层循环 ‘for i in range(n)’ 遍历每一个样本：它提取单个样本 ‘xi’ 和 ‘yi’，计算该样本的误差 ‘error’，并估算出梯度 ‘grad’（注意，此处的梯度仅基于单个样本，计算开销极小）。参数更新 ‘theta -= lr * grad’ 在内层循环中立即执行。在遍历完所有样本后（即一个 ‘epoch’ 结束），代码会计算一次当前 θ 在整个训练集上的 MSE，并将其存入 ‘epoch_losses’ 列表。

（七） 模型评估函数

1. 模型评估封装函数 (evaluate_model)

该函数是一个高级封装，用于一键计算模型在特定数据集上的 MSE 和 MAE 两种性能指标。

```

1  def evaluate_model(X, y, theta):
2      """评估模型性能，返回 MSE 和 MAE。"""
3      preds = X.dot(theta)
4      mse = custom_mean_squared_error(y, preds)
5      mae = custom_mean_absolute_error(y, preds)
6      return mse, mae

```

上述代码接收特征矩阵 ‘X’、真实标签 ‘y’ 和模型参数 ‘theta’ 作为输入。函数的第一步是通过 ‘X.dot(theta)’ 执行矩阵乘法，计算出模型对所有样本的预测值 ‘preds’。随后，它将这些预测值 ‘preds’ 和真实值 ‘y’ 分别传递给 ‘custom_mean_squared_error’ 和 ‘custom_mean_absolute_error’ 这两个底层的指标计算函数（详见后文）。最后，它将计算得到的 ‘mse’ 和 ‘mae’ 两个评估结果打包返回。

2. 均方误差 (MSE) 实现函数 (custom_mean_squared_error)

该函数是 MSE (Mean Squared Error) 的计算函数。

```
1 def custom_mean_squared_error(y_true, y_pred):
2     """自定义实现的均方误差(MSE)。"""
3     return np.mean((y_pred - y_true) ** 2)
```

该函数接收 'y_true' (真实标签) 和 'y_pred' (预测值) 作为输入。它首先通过 'numpy' 的向量化操作计算 'y_pred - y_true'，得到每个样本的预测误差 ('error')。接着，它使用 '** 2' 运算符对所有误差进行逐元素的平方运算 ('error²')。最后，它调用 'np.mean' 函数计算所有这些平方误差的平均值，并返回这个单一的 MSE 数值。

3. 平均绝对误差 (MAE) 实现函数 (custom_mean_absolute_error)

该函数是 MAE (Mean Absolute Error) 的计算函数。

```
1 def custom_mean_absolute_error(y_true, y_pred):
2     """自定义实现的平均绝对误差(MAE)。"""
3     return np.mean(np.abs(y_pred - y_true))
```

该函数同样接收 'y_true' 和 'y_pred' 作为输入，计算 'y_pred - y_true' 得到逐元素的预测误差。与 MSE 不同的是，再调用 'np.abs' 函数来获取每个误差的绝对值 ('|error|')。最后通过 'np.mean' 函数计算所有这些绝对误差的平均值，并返回这个单一的 MAE 数值。

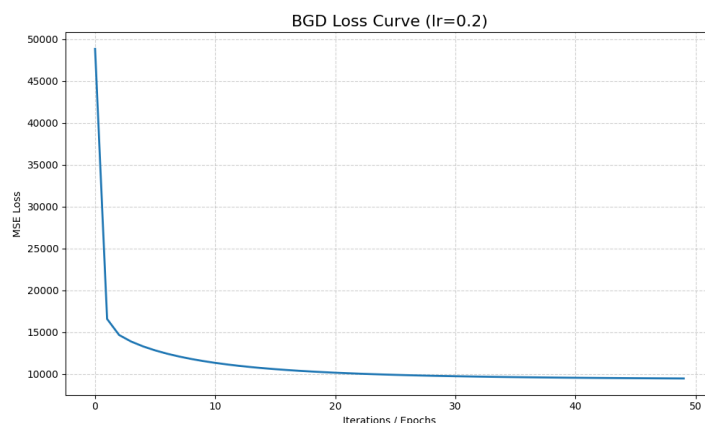
(八) 模型性能对比

现在我们实现了三种多元线性回归的求解方法：正规方程（闭式解）、批量梯度下降（BGD）和随机梯度下降（SGD）。下面我们对每种方法进行对比：

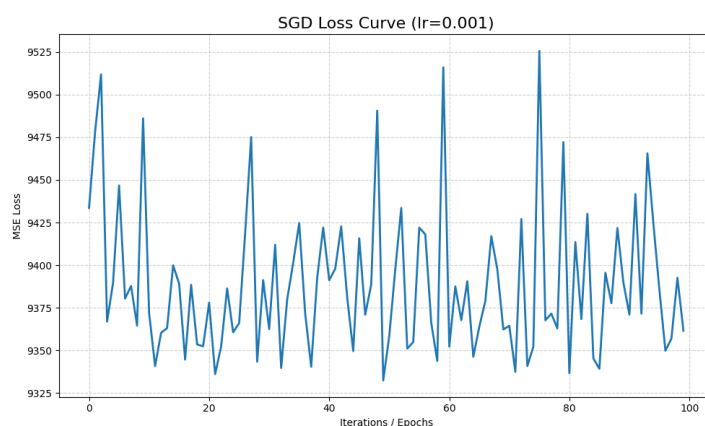
1. 模型训练过程区别

- **正规方程 (Normal Equation):** 正规方程（闭式解）不依赖学习率或迭代次数，而是通过求解线性代数方程 $\theta = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T y$ 来一次性计算出最优的参数 θ 。其优点是无需调参，在特征维度 n 较低时速度很快；但缺点是在特征维度 n 非常高时，计算矩阵的逆 $(\mathbf{X}^T \mathbf{X})^{-1}$ 的时间复杂度（约为 $O(n^3)$ ）会变得极其昂贵。
- **批量梯度下降 (Batch Gradient Descent, BGD):** BGD 在每一次迭代更新 θ 之前，它都必须严格地计算整个训练集（ m 个样本）的总误差和平均梯度。这种方法的优点是收敛路径平稳且精确，在凸优化问题（如本实验）中能稳定地收敛到全局最小值；但缺点是当训练集 m 非常大时，每次迭代的计算成本都极为高昂。
- **随机梯度下降 (Stochastic Gradient Descent, SGD):** SGD 与 BGD 截然不同，SGD 在内层循环中，每处理一个随机样本，就立即计算该样本的梯度并更新一次参数 θ （即一个 epoch 内更新 m 次）。这种方法的优点是训练速度极快，尤其适用于超大规模数据集；但缺点是收敛路径非常嘈杂、振荡剧烈，通常不会精确收敛到最小值，而是在其附近波动。

我们将批量梯度下降（BGD）和随机梯度下降（SGD）训练过程中的损失函数（MSE）与迭代次数（Epochs）之间的关系绘制在图 1 中。



(a) 批量梯度下降 (BGD)



(b) 随机梯度下降 (SGD)

图 1: BGD 与 SGD 损失函数收敛曲线对比

图 1a (上图) 展示了批量梯度下降 (BGD) 的收敛过程。曲线非常平滑，呈现出典型的指数下降趋势。这是因为 BGD 在每一次迭代更新参数时，都计算了**整个训练集**的平均梯度，该梯度精确地指向了损失函数的全局下降方向，因此其收敛路径非常稳定。从图中可以看出，损失 (MSE) 在最初的 10-20 次迭代中迅速下降，随后平稳地收敛到一个稳定的最小值。

图 1b (下图) 展示了随机梯度下降 (SGD) 的收敛过程。与 BGD 形成鲜明对比，SGD 的损失曲线表现出剧烈的振荡和高频噪声。这是因为 SGD 在每次参数更新时，仅使用**单个随机样本**来估计梯度。该梯度具有很高的方差，并不能精确代表整体的下降方向，导致 θ 的更新过程非常嘈杂。尽管曲线振荡，但其整体趋势仍然是下降的，并最终收敛到了与 BGD 相近的损失值，但这证明了其收敛路径远不如 BGD 稳定。

2. 模型训练结果的区别

三种方法的详细性能数据汇总如下表 1 所示。

根据上表 1 的数据，我们可以得出以下结论：正规方程和批量梯度下降 (BGD) 在学习率为 0.2 时的收敛结果几乎完全一致，这证明了 BGD 算法在该学习率下成功收敛到了全局最优解，其测试集 MSE 均为 30455 左右。

与此同时，随机梯度下降 (SGD) 在学习率为 0.001 时，虽然训练集 MSE (9400.9202) 略高于前两者，但其在测试集上的 MSE (29779.2148) 和 MAE (121.1622) 均取得了最优表现。这

表 1 三种回归方法性能对比

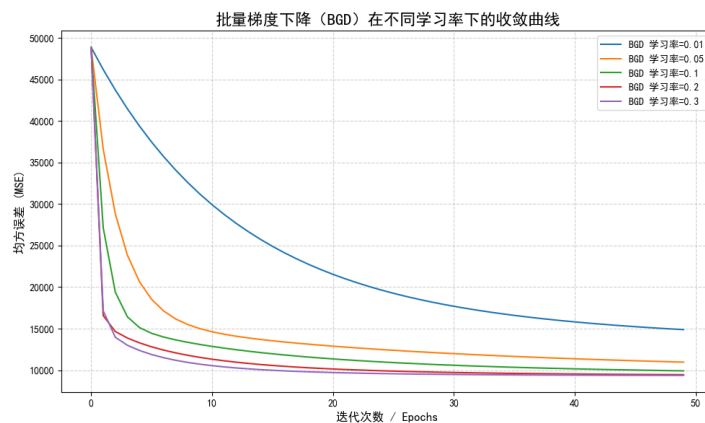
模型方法	学习率	训练集 MSE	训练集 MAE	测试集 MSE	测试集 MAE
正规方程	—	9326.4600	72.9731	30455.9994	122.0254
批量梯度下降 (BGD)	0.2	9326.7153	72.9753	30454.7954	122.0217
随机梯度下降 (SGD)	0.001	9400.9202	74.2002	29779.2148	121.1622

表明 SGD 的随机噪声特性在本任务中可能起到了一种隐式的正则化效果，帮助模型找到了一个泛化能力稍强的参数组合。

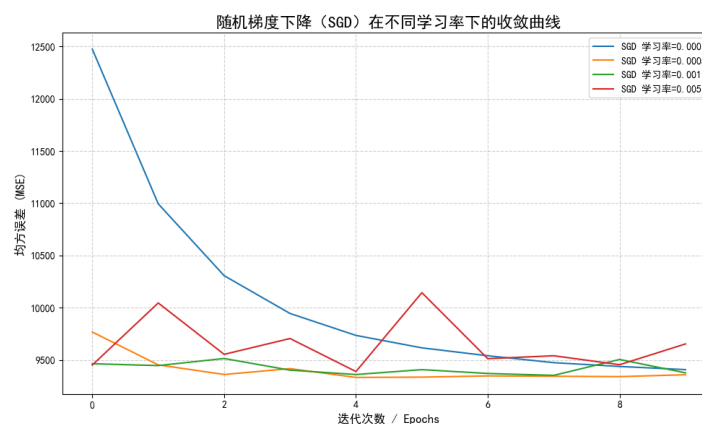
然而，所有模型均表现出训练集 MSE 远低于测试集 MSE 的情况（约 9400 vs 30000），这揭示了我们目前实现的多元线性回归方法存在一定的局限性，存在“过拟合”问题。

3. 学习率对 BGD 与 SGD 模型性能的影响

为了探究不同学习率（Learning Rate, α ）对模型收敛速度和稳定性的影响，我们分别为批量梯度下降（BGD）和随机梯度下降（SGD）设置了一组不同的 α 值进行实验，其损失（MSE）收敛曲线如图 2 所示。



(a) 批量梯度下降（BGD）收敛曲线



(b) 随机梯度下降（SGD）收敛曲线

图 2: BGD 与 SGD 损失函数收敛曲线对比

4. 结果分析

通过对比图 2a（上图）和图 2b（下图），我们观察到两种优化算法对学习率的敏感性存在显著差异。

对于**批量梯度下降（BGD）**（左图），其收敛曲线是平滑且确定性的。这得益于它在每次迭代时都使用了全量数据计算梯度。我们发现，在测试的 α 范围内（0.01 至 0.3），越大的学习率带来了越快的收敛速度。学习率为 0.01 时收敛缓慢，而学习率为 0.3 时则能最快地（在约 20 个 epochs 内）收敛到最低 MSE 水平。

对于**随机梯度下降（SGD）**（下图），其收敛曲线是高度振荡和嘈杂的，这符合其单样本更新的随机特性。SGD 对学习率的设置极其敏感： $\alpha = 0.005$ （红线）显得过大，导致损失函数在迭代过程中出现剧烈尖峰，无法稳定收敛； $\alpha = 0.0001$ （蓝线）则显得过小，虽然稳定，但收敛速度缓慢； $\alpha = 0.0005$ （橙线）和 $\alpha = 0.001$ （绿线）表现最优，它们在极少的迭代次数内（约 2-3 个 epochs）就迅速收敛到了一个较低 MSE 平台。

BGD 适用于较大的学习率且收敛平稳，但收敛至最优值所需的迭代次数（Epochs）更多；SGD 需要使用小得多的学习率，且对学习率的选择非常敏感，过大的 α 会导致其不稳定，但一旦找到合适的 α ，它就能以极快的速度收敛到一个接近最优解的区域。

（九） 初级要求总结

在实验的初级阶段，我们成功实现了三种多元线性回归的求解方法：正规方程、批量梯度下降（BGD）和随机梯度下降（SGD）。

分析对比发现，三种方法在训练机制上各有优劣：正规方程通过矩阵运算一次性求解，无需迭代；BGD 使用全量数据计算梯度，收敛路径平稳但计算量大；SGD 使用单个样本更新参数，训练速度快但收敛过程振荡。

从实验结果上看，正规方程与 BGD（在 $\alpha = 0.2$ 时）收敛结果几乎一致。而 SGD（在 $\alpha = 0.001$ 时）虽然训练误差稍高，但凭借其随机性带来的隐式正则化效果，在测试集上取得了最低的 MSE（29779.2148），泛化能力略优。

通过对不同学习率的分析，我们发现 BGD 对较大的学习率（如 0.3）兼容性好，收敛平稳；而 SGD 对学习率极其敏感，需要设置一个非常小的值才能在快速收敛的同时保持稳定。

四、 中级要求实现

完成初级要求之后，我们进一步进行中级要求。

（一） 特征分析函数 (analyze_coefficients)

为了实现特征重要性排序，我们定义了 ‘analyze_coefficients’ 函数。该函数接收 ‘normal_equation’ 计算得出的 θ （模型系数）和特征名称列表作为输入。

```

1  def analyze_coefficients(theta, feature_names):
2      coef_df = pd.DataFrame({
3          'Feature': feature_names,
4          'Coefficient': theta_flat
5      })
6
7      # 按系数绝对值排序
8      coef_df['Abs_Coefficient'] = coef_df['Coefficient'].abs()
```

```

9         coef_df = coef_df.sort_values(by='Abs_Coefficient', ascending
10                                     =False)
11
12     # 过滤掉 bias (常数项)
13     coef_df_no_bias = coef_df[coef_df['Feature'].str.lower() != '
14         bias']
15
16     print("---特征重要性排序_(按系数绝对值)---")
17     print(coef_df_no_bias.head(10).to_string(index=False))
18
19     # 用于解读的辅助函数
20     def interpret(feature_name, positive_desc, negative_desc):
21         if feature_name not in coef_df['Feature'].values:
22             # ... (省略基准特征的打印) ...
23             return
24
25         coef = coef_df[coef_df['Feature'] == feature_name]['
26             Coefficient'].values[0]
27
28         if coef > 0.1:
29             print(f"-{feature_name}: 系数{coef:+.2f}(正相
30                 关)。")
31             print(f"->解读:{positive_desc}")
32         elif coef < -0.1:
33             print(f"-{feature_name}: 系数{coef:+.2f}(负相
34                 关)。")
35             print(f"->解读:{negative_desc}")
36         else:
37             print(f"-{feature_name}: 系数{coef:+.2f}(影响
38                 微弱)。")

```

上述代码的核心逻辑分为两部分：首先，我们创建一个 ‘DataFrame’ 来存储特征及其对应的 ‘Coefficient’（系数 θ_j ）。通过计算系数的绝对值 ‘Abs_Coefficient’（ $|\theta_j|$ ）并以此降序排序，打印出对模型预测影响幅度最大的前 10 个特征。为了进行对特征重要性排序的简要分析，我们又定义了一个 ‘interpret’ 辅助函数，该函数根据系数的正负（‘coef > 0.1’ 或 ‘coef < -0.1’）来自动打印出预设的正面或负面业务解读文案，方便我们更加详细地进行业务解读。

我们通过在初级要求的基础上添加上述代码，就能够完成中级要求。下面，我们根据程序与运行结果，进行结果分析。

（二） 特征重要性排序

在获得正规方程的解 θ 后，我们通过分析系数 θ_j 的绝对值（ $|\theta_j|$ ）来衡量各特征对模型预测的影响力。系数绝对值越大，代表该特征对租赁量预测结果的影响幅度越大。下表 2 展示了影响力排名前 10 的特征。

表 2 特征重要性排序（按系数绝对值）

特征 (Feature)	系数 (Coefficient)	绝对值 (Abs_Coefficient)
time_period_peak	204.54	204.54
time_period_off_peak	106.96	106.96
yr_1	79.30	79.30
season_4	53.59	53.59
weathersit_3	-48.08	48.08
weathersit_4	-35.29	35.29
atemp	30.30	30.30
season_2	27.35	27.35
temp	25.58	25.58
holiday_1	-22.28	22.28

（三） 关键特征业务解读

基于上表 2 中的模型系数 (Coefficient)，我们对共享单车业务规律进行解读。系数的正负代表了该因素对租赁量的正向/负向影响，系数的绝对值则代表了影响的强度。

1. 环境因素

环境因素是影响用户骑行决策的直接原因。

- **温度 (temp: +25.58)**: 系数为正，说明温度越高，天气越舒适，用户的骑行意愿越强。
- **湿度 (hum: -21.36)**: 系数为负。这表明湿度越高（天气越潮湿或闷热），骑行舒适度下降，租赁量随之减少。
- **风速 (windspeed: -3.50)**: 系数为负，说明风速越高，骑行阻力越大，体感越差，因此会对租赁量产生轻微的负面影响。
- **天气情况 (weathersit):**
 - weathersit_2 (薄雾/阴天: -7.91): 与基准（晴天）相比，阴天的租赁量会轻微下降。
 - weathersit_3 (小雨/小雪: -48.08): 系数为显著负值。这表明下雨或下雪的天气会严重阻碍骑行，导致租赁量大幅减少。

2. 时间因素

时间因素揭示了共享单车需求的核心模式。

- **时间段 (time_period):**
 - time_period_peak (高峰期: +204.54): 这是模型中绝对值最大的系数。它清晰地表明早晚通勤（7-9 点, 16-19 点）是共享单车需求的最核心驱动力，远超其他所有因素。
 - time_period_off_peak (平峰期: +106.96): 白天的平峰期（10-15 点, 20 点）的需求也显著高于基准（深夜和凌晨）。
- **年份 (yr_1: +79.30)**: 2012 年 (yr=1) 的需求显著高于 2011 年 (yr=0, 基准)。这反映了共享单车业务在该年度的快速增长和市场渗透。

- 季节 (season):

- season_2 (夏天: +27.35) 和 season_4 (冬天: +53.59): 这两个系数均高于基准 (春天)。夏天需求更高是合理的, 并且夏天的温度会更高, 也同时符合温度参数的因素。但是按照温度对于预测值的影响, 冬天的需求应该会是一年中较低的, 但是冬天需求要高于春天, 这可能是因为有其他因素的干预, 导致模型将温度的影响剥离出去了, 春季可能多雨且气温不稳定, 导致其骑行适宜度反而不如干冷冬天。

3. 其他因素

- 节假日 (holiday_1: -22.28): 系数为负, 说明节假日时, 绝大多数通勤需求消失了, 因此即使天气晴朗, 总租赁量也会显著下降。
- 工作日 (workingday_1: +0.25): 系数接近于 0, 影响微弱。这表明“是否为工作日”这个信息本身几乎是多余的, 因为“工作日的需求”这个概念已经被 time_period_peak (高峰期) 特征完全吸收了。

(四) 特征交互作用

为了探索不同特征组合对租赁量的复杂影响, 我们在数据预处理阶段手动创建了三组交互特征。这通过在 'preprocess_with_interactions' 函数中, 将相关的特征列相乘来实现。

```

1      # 1. 温度 x 假期 (假期的休闲用户是否对温度更敏感?)
2      if 'holiday_1' in X_df.columns:
3          X_df['temp_x_holiday'] = X_df['temp'] * X_df['holiday_1']
4
5      # 2. 坏天气 x 高峰期 (高峰时段下雨/雪是否导致需求锐减?)
6      if 'weathersit_3' in X_df.columns and 'time_period_peak' in X_df.
7          columns:
8          X_df['weathersit_x_peak'] = X_df['weathersit_3'] * X_df['
9              time_period_peak']
10
11     # 3. 湿度 x 工作日 (工作日的通勤者是否对湿度容忍度更高?)
12     if 'workingday_1' in X_df.columns:
13         X_df['hum_x_workingday'] = X_df['hum'] * X_df['workingday_1']

```

上述代码片段在独热编码之后执行。我们首先检查所需的原始特征列是否存在, 然后通过简单的乘法操作 'X_df['temp'] * X_df['holiday_1']' 创建一个新的特征列 'temp_x_holiday'。这个新特征的值仅在“既是假日又是某个特定温度”时才不为零, 从而允许模型学习温度在假日与非假日下的不同影响。其他两个交互项 (坏天气 × 高峰期, 湿度 × 工作日) 也通过类似的方式创建。

(五) 统计显著性 (P 值) 分析

在评估特征 (包括主效应和交互效应) 对共享单车租赁量影响的可靠性时, 我们需要判断观测到的系数是否“真实存在”, 还是仅仅由数据的随机波动引起。统计显著性检验, 特别是 P 值的计算, 就是回答这个问题的核心工具。

1. P 值的基本概念

P 值 (P-value) 是在“零假设” (Null Hypothesis, H_0) 为真的前提下, 观测到当前样本结果的概率。在我们的线性回归模型中, 对于每一个特征 j , 其零假设 H_0 通常是:

$$H_0 : \theta_j = 0$$

即假设该特征的真实系数为 0, 它对目标变量 y (租赁量) 没有任何实际影响。

P 值衡量的是反对 H_0 的证据强度。我们通常设定一个显著性水平 α (例如 0.05):

- 如果 P 值 $< \alpha$ (例如 $P < 0.05$): 这表示, 如果该特征真的无效 (H_0 为真), 那么我们观测到当前这么大的系数 (或更大) 的概率非常小 (小于 5%)。我们因此拒绝零假设 H_0 , 认为该特征的影响是**统计显著的 (statistically significant)**。
- 如果 P 值 $\geq \alpha$ (例如 $P \geq 0.05$): 这表示, 即使该特征真的无效, 我们也有较大的概率 (大于等于 5%) 观测到当前的系数。我们因此**没有足够的证据**拒绝零假设 H_0 , 认为该特征的影响是**统计不显著的**。

2. P 值的计算实现

为量化每个特征 (包括交互特征) 对租赁量影响的统计确定性, 我们手动实现了 P 值的计算流程, 其核心逻辑位于 ‘analyze_coefficients_numpy’ 函数中。

```

1      def analyze_coefficients_numpy(theta, inv_XTX, X_train, y_train,
2          feature_names):
3          # ... (省略 1. 计算基本参数 n, k, dof) ...
4
5          # 2. 计算残差
6          y_pred = X_train.dot(theta)
7          residuals = y_train - y_pred
8
9          # 3. 计算残差平方和 (RSS)
10         rss = residuals.T.dot(residuals)[0, 0] # 得到一个标量
11
12         # 4. 估计误差方差
13         sigma_squared_hat = rss / dof
14
15         # 5. 计算系数的方差-协方差矩阵
16         var_cov_matrix = sigma_squared_hat * inv_XTX
17
18         # 6. 提取标准误 (SE)
19         coef_variances = np.diag(var_cov_matrix)
20         std_errors = np.sqrt(coef_variances)
21
22         # 7. 计算 t-统计量
23         t_stats = theta_flat / std_errors
24
25         # 8. 计算 P-值 (依赖 scipy.stats)
26         p_values = 2 * sps.t.sf(np.abs(t_stats), df=dof)

```


在上述代码中，我们使用了最小二乘法（OLS）：首先计算模型的残差（‘residuals’）和残差平方和（‘rss’）；然后利用 ‘rss’ 和自由度 ‘dof’ 估计误差的方差 $\hat{\sigma}^2$ （‘sigma_squared_hat’）；我们再通过 $\hat{\sigma}^2$ 和 ‘normal_equation_with_stats’ 函数返回的 $(\mathbf{X}^T \mathbf{X})^{-1}$ （‘inv_XTX’）计算出系数的方差-协方差矩阵 ‘var_cov_matrix’；从该矩阵的对角线提取方差并开方，得到每个系数的标准误 ‘std_errors’；利用系数估计值 $\hat{\theta}$ 和标准误计算 t-统计量 ‘t_stats’ = $\hat{\theta}/SE(\hat{\theta})$ ；最后，调用 ‘scipy.stats.t.sf’（t 分布的生存函数，即 1-CDF）并乘以 2，计算出双尾检验的 P 值 ‘p_values’。这些 P 值随后被用于判断每个特征的统计显著性。

3. 结果分析

我们通过手动实现的 OLS 统计推断，得到了包含 P 值的详细分析结果，如下表 3 所示（按系数绝对值排序）。

表 3 特征统计显著性分析结果

特征 (Feature)	系数 (Coef.)	标准误 (Std Err)	t 统计量 (t)	P 值 (P> t)	绝对系数	显著性
time_period_peak	210.4228	2.2898	91.8970	0.0000	210.4228	***
time_period_off_peak	107.4023	2.3176	46.3416	0.0000	107.4023	***
hum_x_workingday	86.4730	9.5857	9.0210	0.0000	86.4730	***
yr_1	79.2203	2.2040	35.9447	0.0000	79.2203	***
temp_x_holiday	77.8944	26.2800	2.9640	0.0030	77.8944	**
weathersit_x_peak	-58.5491	6.5430	-8.9483	0.0000	58.5491	***
holiday_1	-58.5362	12.7046	-4.6075	0.0000	58.5362	***
workingday_1	-53.8438	6.3024	-8.5434	0.0000	53.8438	***
season_4	52.5240	3.0227	17.3767	0.0000	52.5240	***
weathersit_4	-38.4895	55.4990	-0.6935	0.4880	38.4895	
hum	-32.9369	1.7156	-19.1979	0.0000	32.9369	***
atemp	32.9222	7.6669	4.2940	0.0000	32.9222	***
bias	31.9610	3.0621	10.4376	0.0000	31.9610	***
weathersit_3	-30.1026	4.0684	-7.3992	0.0000	30.1026	***
season_2	28.5105	2.7826	10.2459	0.0000	28.5105	***
temp	22.2001	7.8432	2.8305	0.0047	22.2001	**
weathersit_2	-8.4138	2.1685	-3.8799	0.0001	8.4138	***
windspeed	-3.2173	0.9896	-3.2510	0.0012	3.2173	**
season_3	1.3073	4.1527	0.3148	0.7529	1.3073	

表 4 显著性代码: 0 '***' 0.001 '**' 0.01 '*' 0.05

根据上表 3 的结果，我们可以得出以下关键分析：

- **绝大多数特征高度显著：**除了 ‘weathersit_4’（恶劣天气）和 ‘season_3’（秋天）之外，所有特征的 P 值都远小于 0.05（大部分甚至小于 0.001），这表明它们对共享单车租赁量的影响是极其显著的，几乎不可能由随机巧合造成的。
- **不显著特征的确认：**
 - **weathersit_4**（极端天气）的 P 值为 0.4880，远大于 0.05，所以该特征的系数（-38.4895）很可能只是噪音。这通常是由于该事件（极端天气）在数据集中发生次数过少，导致模型无法学习到一个稳定可靠的影响。

– `season_3` (秋天) 的 P 值为 0.7529, 同样远大于 0.05。这表明, 在模型考虑了温度等其他因素后, 秋季的租赁量与基准季节 (春季) 相比, 并无统计上的显著差异。

- **交互效应的显著性:** 三个交互特征 (`'hum_x_workingday'`, `'temp_x_holiday'`, `'weathersit_x_peak'`) 的 P 值都极小 (均小于 0.003), 这强烈证明了引入这些交互项是正确且有价值的。这三个交互特征捕捉到了主效应 (温度、天气) 在不同情境下 (节假日、高峰期) 的差异化影响, 显著提升了模型的解释力。

(六) 中级要求总结

在中级要求阶段, 我们深入分析了基础线性回归模型的内在机制与业务含义。通过对正规方程解出的系数进行排序, 我们发现 `'time_period_peak'` (高峰期)、`'time_period_off_peak'` (平峰期) 和 `'yr_1'` (年份) 是对共享单车租赁量影响最大的三个因素。

结合业务知识对模型系数进行解读, 我们可以发现温度、湿度、天气状况等环境因素对用户骑行意愿的直观影响, 同时也发现了节假日因通勤需求消失而导致租赁量下降等规律。

为了捕捉更复杂的现实模式, 我们手动构建了三组交互特征 (温度 $\times$ 假期、坏天气 $\times$ 高峰期、湿度 $\times$ 工作日)。通过手动实现的 P 值计算和统计显著性分析, 我们验证了这些交互项以及绝大多数主效应特征 (除 `'weathersit_4'` (恶劣天气) 和 `'season_3'` (秋季) 外) 都具有高度的统计显著性 (P 值远小于 0.05)。这不仅证实了我们模型捕捉到的规律是可靠的, 也证明了引入交互特征能够显著提升模型的解释力和对现实复杂性的拟合能力。

五、 高级要求

在初级和中级要求的基础上, 我们引入了正则化技术 (岭回归和 LASSO) 来提高模型的泛化能力并进行特征选择, 同时使用交叉验证来自动选择最优的正则化参数。

在完成上述进阶任务之后, 我们还能够通过将模型性能的可视化来对模型进行更加深入的理解与分析。

(一) 原理介绍

1. 岭回归 (Ridge)

岭回归是线性回归的一种正则化形式, 它在标准线性回归的成本函数 $J_{OLS}(\theta) = \frac{1}{m} \sum (\hat{y}^{(i)} - y^{(i)})^2$ 的基础上, 增加了一个 L2 正则化项 (Ridge Penalty Term)。该项惩罚模型系数 θ 的平方和, 旨在防止系数过大, 从而降低模型复杂度, 减轻过拟合。其成本函数为:

$$J_{Ridge}(\theta) = J_{OLS}(\theta) + \alpha \sum_{j=1}^n \theta_j^2$$

其中 $\alpha \geq 0$ 是正则化强度超参数, 控制惩罚的大小; n 是特征数量 (不包括偏置项 θ_0 , 通常不对其进行惩罚)。岭回归仍然有闭式解, 其正规方程解为:

$$\theta = (\mathbf{X}^T \mathbf{X} + \alpha \mathbf{I})^{-1} \mathbf{X}^T y$$

其中 $\mathbf{I}$ 是一个 $n \times n$ 的单位矩阵, 但其左上角第一个元素为 0, 以确保不对偏置项 θ_0 施加惩罚。

2. LASSO 回归

LASSO (Least Absolute Shrinkage and Selection Operator) 回归是另一种线性回归的正则化形式。它与岭回归类似，但使用的是 L1 正则化项 (LASSO Penalty Term)，即惩罚模型系数 θ 的绝对值之和。其成本函数为：

$$J_{LASSO}(\theta) = J_{OLS}(\theta) + \alpha \sum_{j=1}^n |\theta_j|$$

L1 正则化的一个关键特性是它具有**稀疏性 (Sparsity)**：当 α 足够大时，LASSO 倾向于将某些特征的系数精确地压缩到零，从而实现自动的特征选择。与岭回归不同，LASSO 的成本函数由于绝对值项的存在而不可导，因此**没有闭式解**。它必须使用迭代优化算法求解，例如坐标下降法 (Coordinate Descent) 或 LARS 算法。坐标下降法的核心是利用软阈值算子 (Soft Thresholding Operator) 逐个更新系数。

3. K-Fold 交叉验证

K-Fold 交叉验证 (K-Fold Cross-Validation) 是一种常用的模型评估和超参数选择技术，用于评估模型在未知数据上的泛化能力。其基本流程如下：

1. 将训练数据集随机划分为 K 个大小相似的互斥子集（称为“折”，Fold）。
2. 进行 K 次迭代：在第 k 次迭代中，将第 k 折作为验证集，其余 $K - 1$ 折合并作为训练集。
3. 在该训练集上训练模型，并在验证集上评估性能指标（例如 MSE）。
4. K 次迭代完成后，将 K 个验证集上的性能指标取平均值，作为该模型（或该组超参数）的最终评估分数。

我们使用 K-Fold 交叉验证来为岭回归和 LASSO 选择最优的正则化强度超参数 α 。即对一系列候选 α 值，分别执行 K-Fold 交叉验证，最终选择那个平均验证 MSE 最小的 α 作为最佳参数。

(二) 代码实现

1. 岭回归 (Ridge)

我们使用正规方程手动实现了岭回归的求解。

```

1      def ridge_regression_normal(X, y, alpha):
2          n, m = X.shape
3
4          # 创建单位矩阵 I'
5          I_prime = np.identity(m)
6          # 不对 bias (j=0) 进行惩罚
7          I_prime[0, 0] = 0.0
8
9          # (X.T * X + alpha * I')
10         XTX = X.T.dot(X)
11         penalty_term = alpha * I_prime
12
13         try:
14             # (X.T * X + alpha * I')^(-1)

```

```

15         inv_term = np.linalg.inv(XTX + penalty_term)
16     except np.linalg.LinAlgError:
17         inv_term = np.linalg.pinv(XTX + penalty_term)
18
19         # theta = inv * X.T * y
20         theta = inv_term.dot(X.T).dot(y)
21     return theta

```

‘ridge_regression_normal’ 函数精确地实现了岭回归的闭式解。我们先创建一个调整后的单位矩阵 ‘I_prime’，将其第一个对角元素（对应偏置项）设为 0。然后计算 $\mathbf{X}^T\mathbf{X} + \alpha\mathbf{I}'$ ，并尝试使用 ‘np.linalg.inv’ 计算其逆矩阵。为了处理可能出现的奇异矩阵情况（虽然 L2 正则化通常会改善矩阵的条件数），代码包含了 ‘try...except’ 块，在求逆失败时转而使用伪逆 ‘np.linalg.pinv’。最后，它完成矩阵乘法得到最优参数 θ 。

2. LASSO 回归

由于 LASSO 没有闭式解，我们手动实现了坐标下降（Coordinate Descent）算法来求解。该算法的核心是软阈值算子。

```

1     def soft_thresholding(rho, lambda_):
2         if rho > lambda_:
3             return rho - lambda_
4         elif rho < -lambda_:
5             return rho + lambda_
6         else:
7             return 0.0
8
9     def lasso_coordinate_descent(X, y, alpha, epochs=1000, tol=1e-5):
10         n, m = X.shape
11         y_flat = y.ravel()
12         theta = np.zeros(m)
13         z = (2.0 / n) * np.sum(X**2, axis=0) # 预计算用于归一化的项
14         z[z == 0] = 1.0
15
16         for epoch in range(epochs):
17             theta_old = theta.copy()
18             for j in range(m): # 逐个坐标（系数）更新
19                 y_pred = X.dot(theta)
20                 r_j = y_flat - y_pred + X[:, j] * theta[j] #
21                     计算不含第 j 项的残差
22                 rho_j = (2.0 / n) * np.dot(X[:, j], r_j) # 计
23                     算相关项
24                 lambda_ = alpha
25
26                 if j == 0: # 不惩罚 bias 项
27                     theta[j] = rho_j / z[j]
28                 else: # 对其他特征项应用软阈值
29                     theta[j] = soft_thresholding(rho_j, lambda_)
30                     / z[j]

```

```

28
29         # 检查收敛
30         if np.linalg.norm(theta - theta_old) < tol:
31             break
32
33     return theta.reshape(-1, 1)

```

‘lasso_coordinate_descent’ 函数通过迭代优化求解 LASSO 问题。在每次 ‘epoch’ 中，它会遍历所有特征（‘for j in range(m)’）。对于每个特征 j ，首先计算在固定其他系数时，仅与 θ_j 相关的残差项 r_j 和相关项 ρ_j 。如果 $j = 0$ （偏置项），则直接更新；否则，将 ρ_j 和正则化强度 α （代码中为 ‘lambda_’）传入 ‘soft_thresholding’ 函数。该函数执行软阈值操作：如果 ρ_j 的绝对值小于 α ，则将 θ_j 设为 0（实现稀疏性）；否则，将其向 0 收缩 α 的量。‘z’ 数组是预计算的归一化项，用于确保更新步长的正确性。迭代过程持续进行，直到系数向量 θ 的变化小于容忍度 ‘tol’ 或达到最大迭代次数 ‘epochs’。

3. K-Fold 交叉验证

我们手动实现了 K-Fold 交叉验证的流程，用于在给定的一系列 α 值中为 Ridge 和 LASSO 选择最优参数，由于我们在实验一中已经完成了 K-Fold 交叉验证的实现，我们在写代码时可以参考实验一中代码的思路。

在我们的代码中，我们使用 ‘k_fold_cross_validation’ 函数实现了完整的 K 折交叉验证逻辑。它首先打乱训练数据 ‘X_train’，‘y_train’ 以消除顺序依赖。然后，它循环 K 次（‘k_folds’）：在每次循环中，它将数据划分为临时的训练子集（‘X_sub_train’，‘y_sub_train’）和验证集（‘X_val’，‘y_val’）。对于每一个待测试的超参数 α （‘alphas’ 列表），它使用当前的训练子集来训练 Ridge 或 LASSO 模型，并在验证集上计算 MSE。这个 MSE 被记录下来。完成 K 折循环后，函数计算每个 α 在 K 个验证集上的平均 MSE，并返回那个使得平均 MSE 最小的 ‘best_alpha’。

（三） 初步结果与可视化分析

在选择了最优 α 并训练最终的模型后，我们对其在测试集上的预测效果进行了诊断，结果如图 3 和图 4 所示。

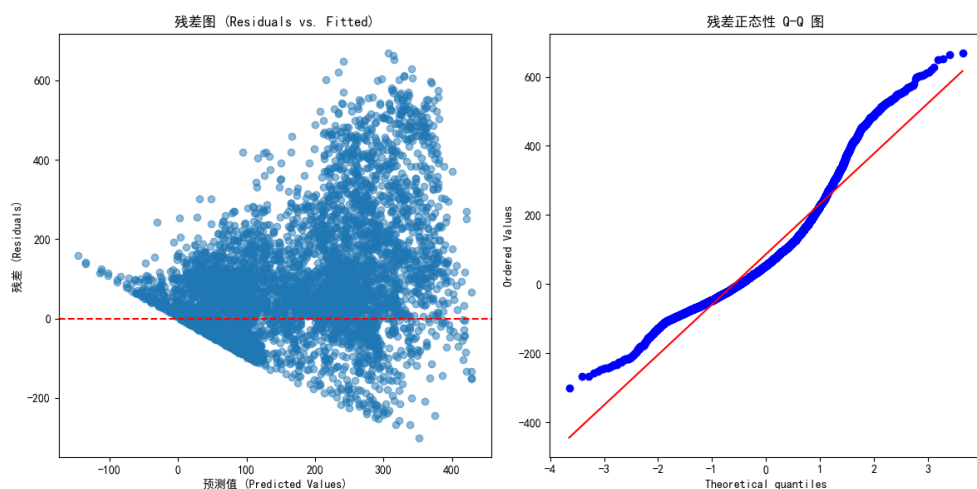


图 3: 模型诊断图 (初步结果)

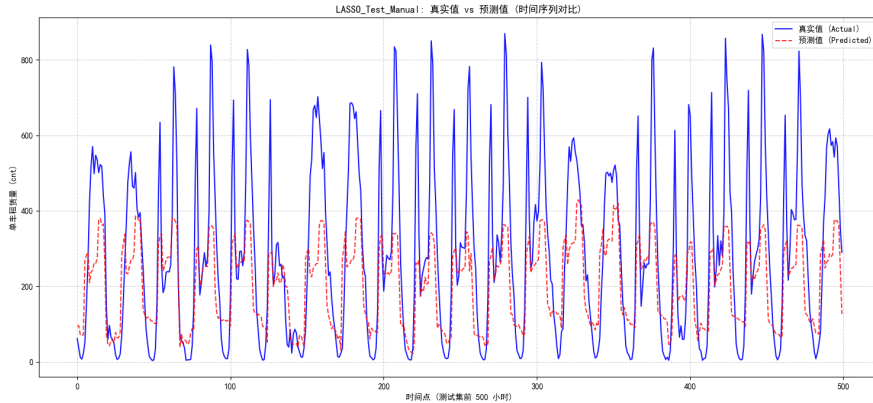


图 4: 时间序列预测效果 (初步结果)

1. 诊断图分析 (图 3)

图 3 展示了模型在测试集上的残差分析结果。

- **残差图 (左图: Residuals vs. Fitted):** 该图清晰地呈现出一个“喇叭口”或扇形形状。当预测值（横轴）较低时（例如 0-100），残差（纵轴，即真实值-预测值）紧密地聚集在 0 附近，表明模型在预测低需求时段时误差较小。然而，随着预测值的增大（例如 200-400），残差的散布范围急剧扩大，出现了大量较大的正残差（高达 600）。这表明模型误差的方差不是恒定的，存在严重的**异方差性 (Heteroscedasticity)**，违反了线性回归的基本假设。
- **Q-Q 图 (右图):** 该图用于检验残差是否符合正态分布。理想情况下，蓝色的数据点应紧密贴合在红色的理论线上。然而，图中显示蓝点在两端（尤其是右上尾部）明显偏离红线，呈现“S”形。这表明残差并不服从正态分布，特别是存在比正态分布更“重”的正向尾部，对应于残差图中大量的正向大误差。

2. 时间序列预测效果分析 (图 4)

图 4 直观地展示了模型预测（红色虚线）与真实租赁量（蓝色实线）在测试集前 500 个时间点上的对比。

- **优点:** 模型成功捕捉到了需求的周期性模式。预测曲线能够准确地跟随真实曲线的起伏节奏，识别出高峰和低谷出现的时间点。
- **缺点:** 模型严重低估了峰值需求。当真实租赁量（蓝线）飙升至 800 甚至 900 时，模型的预测值（红线）往往被“削平”在 400-450 左右。这与残差图中观察到的大量正误差（真实值远大于预测值）完全吻合。模型似乎只学会了“何时”会出现高峰，但无法准确预测“高峰有多高”。

3. 初步结论

虽然 LASSO 模型（包括 OLS 和 Ridge，因其性能相似）能够学习到影响需求的关键因素和时间模式，但在原始尺度上直接应用线性回归，由于数据本身的特性（偏态分布、方差随均值变化），导致模型严重违反统计假设，无法准确拟合需求的峰值，表现出明显的异方差性和对高峰需求的系统性低估。

这些显著的统计问题和业务上的预测缺陷（特别是对高峰的低估）表明，直接在原始尺度上应用线性回归模型是不够的，我们需要进一步的优化策略来改善模型在需求高峰时刻的拟合能力。

（四） 进一步优化

1. 对数变换原理

对数变换是一种常用的方差稳定变换，特别适用于处理正值且右偏分布的数据（如计数数据）。其原理如下：

- **压缩大值，拉伸小值：**对数函数 $f(x) = \log(x)$ 增长缓慢。它对较大的 y 值（如高峰需求）的压缩作用远强于对较小的 y 值（如低谷需求）。
- **改善分布：**这种非线性压缩使得变换后的变量 $\log(y)$ 的分布更接近对称的正态分布。
- **稳定方差：**当原始数据的方差随均值增大时，对数变换通常能使变换后数据的方差变得更加恒定（即更接近同方差性）。
- **处理零值：**由于 $\log(0)$ 未定义，而租赁量 ‘cnt’ 可能为 0，我们实际使用的是 $\log(1 + y)$ 变换，对应 ‘numpy’ 中的 ‘np.log1p()’ 函数。
- **逆变换：**模型在 $\log(1 + y)$ 尺度上进行预测后，需要通过逆函数 $e^x - 1$ （对应 ‘numpy’ 中的 ‘np.expm1()’ 函数）将预测结果转换回原始的 ‘cnt’ 尺度，以便进行业务解释和评估。

通过这种变换，我们期望模型能够更好地满足线性回归的统计假设，从而更准确地拟合数据，特别是改善对高峰需求的预测。

2. 目标变量变换

在数据预处理之后，代码立即对训练集和测试集的目标变量应用 ‘np.log1p’ 变换。

```
1 y_train_log = np.log1p(y_train_orig)
2 y_test_log = np.log1p(y_test_orig)
```

此步骤将原始的租赁量 ‘y_train_orig’ 和 ‘y_test_orig’ 转换为对数尺度下的 ‘y_train_log’ 和 ‘y_test_log’。后续所有的模型训练和交叉验证都将使用这些变换后的 ‘_log’ 变量作为目标。

3. 模型训练与交叉验证

所有的模型训练函数（包括 ‘normal_equation’、‘ridge_regression_normal’、‘lasso_coordinate_descent’）以及交叉验证函数 ‘k_fold_cross_validation’ 现在都接收 ‘y_train_log’ 作为输入。

```
1 # OLS 训练
2 theta_ne_log = normal_equation(X_train, y_train_log)
3
4 # 交叉验证（以 LASSO 为例）
5 best_alpha_lasso = k_fold_cross_validation(
6     'lasso', X_train, y_train_log, alphas_to_try, k_folds=5 # 使用
7     y_train_log
8 )
```

```

9      # 最终 LASSO 训练
10     theta_lasso_log = lasso_coordinate_descent(X_train, y_train_log,
        best_alpha_lasso, ...)

```

这意味着模型现在学习的是特征 $\mathbf{X}$ 与 $\log(1+y)$ 之间的线性关系。交叉验证评估 MSE 也是在 ‘log’ 尺度上进行的，以找到在该尺度下表现最好的 α 。

4. 预测与逆变换

模型直接预测出的结果是在对数尺度上的 ($\hat{y}_{\log}$)。为了得到业务上有意义的预测值（预测的租赁量 $\hat{y}$ ），需要应用逆变换 ‘np.expml’。

```

1      # 预测 log 尺度结果 (以 LASSO 为例)
2      y_pred_test_log_lasso = X_test.dot(theta_lasso_log)
3
4      # 逆变换回原始尺度
5      y_pred_test_final_lasso = np.expml(y_pred_test_log_lasso)
6
7      # 裁剪可能的负值 (数值精度问题可能导致极小的负值)
8      y_pred_test_final_lasso[y_pred_test_final_lasso < 0] = 0

```

此步骤将模型输出的 $\log(1+\hat{y})$ 转换回 $\hat{y}$ 。添加了一个裁剪步骤以确保最终预测值非负。

5. 性能评估

模型的最终性能 (MSE, MAE) 必须在原始尺度上进行评估，因为业务关心的是对实际租赁量的预测误差。

```

1      # 使用原始 y_test_orig 和逆变换后的 y_pred_test_final_lasso
2      lasso_test_mse = custom_mean_squared_error(y_test_orig,
        y_pred_test_final_lasso)
3      lasso_test_mae = custom_mean_absolute_error(y_test_orig,
        y_pred_test_final_lasso)

```

这一步确保了我們比较的是模型在实际业务问题上的表现。

(五) 优化结果与可视化分析

在应用了对数变换 ($\log(1+y)$) 并重新训练 LASSO 模型（使用交叉验证选择的最佳 α ）后，我们对优化后的模型进行了诊断和效果评估。

1. 模型诊断 (Log 尺度)

图 5 展示了优化后模型在对数尺度上的残差诊断结果。

与原始尺度下的诊断图（图 3）相比，我们观察到显著的改善：

- **残差图 (左图)：**原始模型中明显的“喇叭口”形状（异方差性）已基本消失。现在的残差点在 0 刻度线上下分布得更加均匀和随机，表明误差的方差趋于恒定，更符合线性回归的假设。虽然在预测值较低（横轴约 2-3）时似乎仍有一些结构性偏差（残差偏正），但整体改善巨大。

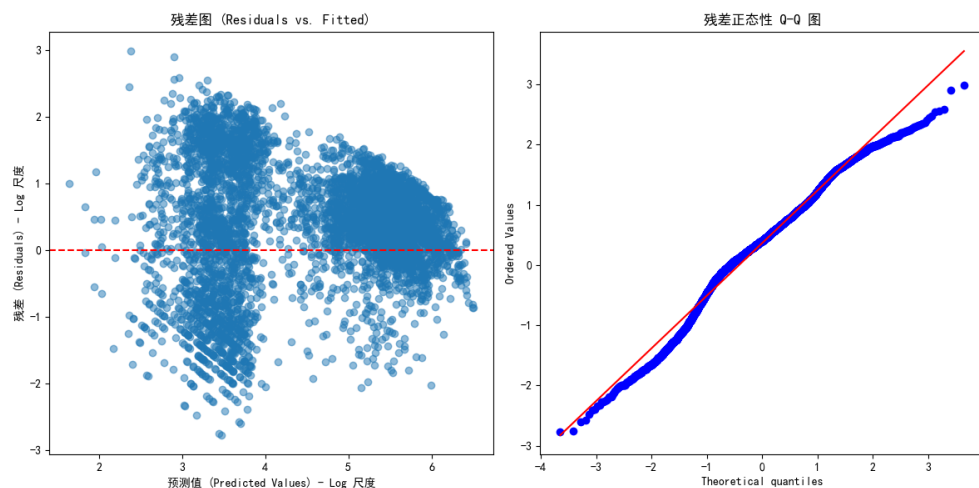


图 5: 模型诊断图 (对数尺度)

- **Q-Q 图 (右图):** 蓝色的数据点现在非常紧密地贴合在红色的理论线上, 仅在最高值处略有偏离。这表明经过对数变换后, 模型的残差分布已非常接近正态分布, 远优于原始模型中严重的“S”形偏离。

诊断结果证实, 对数变换成功地解决了原始模型违反统计假设的核心问题, 使得模型预测更加准确。

2. 时间序列预测效果 (原始尺度)

图 6 展示了优化后模型在原始尺度上的时间序列预测效果。

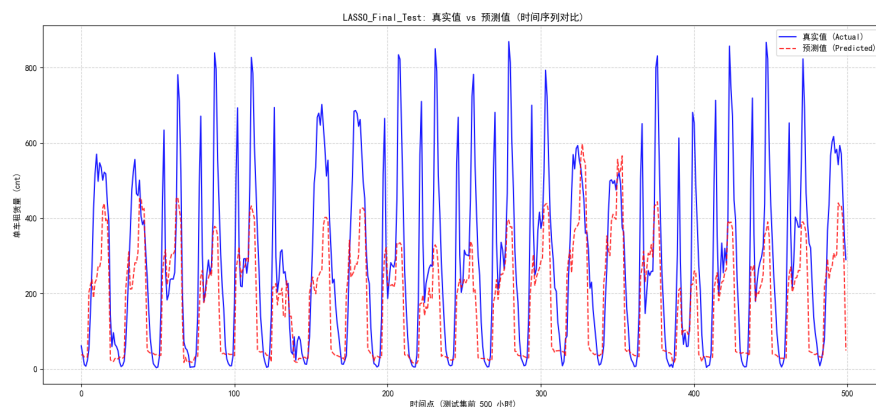


图 6: 时间序列预测效果 (优化后)

与原始模型的预测图 (图 4) 相比, 优化后的预测 (红色虚线) 在拟合高峰需求方面取得了显著进步:

- **峰值预测改善:** 原始模型预测的峰值被“削平”在 400-450 左右, 而新模型现在能够预测到更高的峰值, 例如在多个高峰期预测值达到了 600-700。虽然与 800-900 的真实峰值仍有差距, 但“严重低估”的问题已得到大幅缓解。
- **周期性捕捉依旧准确:** 模型仍然能够准确地捕捉需求的周期性模式。

3. 性能指标分析

尽管优化后的模型在 Test MSE/MAE 指标上略有上升（LASSO 的 Test MSE 从 30215 升至 33194），但结合诊断图和时间序列图的分析，我们可以发现这个模型的预测准确度反而更高。MSE/MAE 的上升可能是因为新模型在尝试拟合高峰时引入了新的“高估”误差（新模型再低谷时误差不大），但这远比旧模型系统性地“低估”所有高峰要好（旧模型在高峰时误差很大），所以新模型对业务的实际指导价值更高。

4. LASSO 特征选择

在对数尺度上训练时，LASSO 模型成功将 ‘weathersit_4’（极端天气）的系数压缩为零，实现了自动特征选择，这进一步验证了该特征的不稳定性以及在优化后的数据尺度上进行正则化的优势。

尽管在原始尺度的 MSE/MAE 指标上有所牺牲，但优化后的模型在统计健康度和业务预测能力上均获得了显著提升。

（六） 高级要求总结

在高级要求阶段，我们首先手动实现了岭回归（Ridge）和 LASSO 两种正则化线性模型。岭回归通过 L2 范数惩罚系数平方和来防止过大系数，我们使用其正规方程解实现。LASSO 通过 L1 范数惩罚系数绝对值和，能够将部分系数压缩至零实现特征选择，我们使用坐标下降法和软阈值算子迭代求解。为了自动选择最优的正则化强度 α ，我们还手动实现了 K-Fold 交叉验证流程。

初步应用这些模型（在原始 ‘cnt’ 尺度上训练）后，模型诊断（残差图和 Q-Q 图）暴露出严重的异方差性和非正态残差问题，时间序列可视化也显示模型严重低估了需求的峰值。

针对这些问题，我们采用了对目标变量 ‘cnt’ 进行对数变换（ $\log(1+y)$ ）的优化策略。在对数尺度上重新训练模型并进行交叉验证后，新的诊断图显示异方差性和非正态残差问题得到了显著改善，模型更符合统计假设。同时，将预测结果逆变换回原始尺度后，时间序列可视化显示模型对高峰需求的预测能力大幅提升。在对数尺度下，LASSO 也成功地自动移除了一个先前被识别为不稳定的特征（‘weathersit_4’）。尽管优化后的模型在原始尺度的 MSE/MAE 指标上略有增加，但其统计健康度和业务预测能力的提升证明了对数变换优化的有效性。

六、 实验总结

本次实验，我们成功完成了初级、中级与高级的要求，并对多元线性规划的整体思路、优化方法与具体实现有了全面的了解。

初级阶段，我们成功构建了多元线性回归模型，实现了数据预处理（包括时间序列划分、三级时段划分、标准化等功能），并实现了正规方程、BGD 和 SGD 三种求解方法。对比发现，SGD 在测试集上性能略优，但所有模型均显示训练与测试误差差距较大。

中级阶段，通过特征重要性排序和系数解读，确认了高峰时段、年份等核心驱动因素，并验证了天气、节假日等影响。为捕捉复杂模式，引入了交互特征，并通过手动实现的 P 值计算，证实了大部分特征及交互项的统计显著性。

高级阶段，我们手动实现了岭回归、LASSO 及 K-Fold 交叉验证。然而，对初步模型的诊断暴露出严重的异方差性、非正态残差，以及对需求峰值的严重低估。为了解决这一问题，我们采用了对目标变量 ‘cnt’ 进行对数变换（ $\log(1+y)$ ）的优化策略。优化后，模型诊断结果显示统计

问题显著改善，时间序列可视化证明模型对高峰需求的预测能力大幅提升。同时，LASSO 在对数尺度下成功移除了不显著特征 ‘weathersit_4’（恶劣天气）。

本实验系统地实践了线性回归及其正则化、优化的流程，证明了对数变换在处理此类计数数据时的有效性，最终构建了一个预测能力与业务预测能力较强的共享单车需求预测模型。

项目的完整实验报告与源代码已同步至 GitHub, 详情请见:<https://github.com/fan-tuaner614/Machine-Learning>